

Do Zero ao App Pronto

O Guia Prático Passo a Passo do Ecossistema AIDD

Ecossistema AIDD — Engenharia Canônica

06 de outubro de 2026

Resumo Este mini-livro apresenta o roteiro prático e determinístico de engenharia de software dentro do **Ecossistema AIDD**, cobrindo cada etapa desde o acesso inicial ao repositório monorepo até a execução de uma aplicação moderna e funcional no navegador.

A narrativa utiliza a abordagem dual canônica: **Na Festa** (metáfora pedagógica e lúdica que explica o propósito de cada fase como se fosse o preparo de um banquete) e **Na Casa** (o rigor técnico estrito, comandos de terminal invariantes, Git Worktrees efêmeras, testes sem mocks, 65 Quality Gates globais e o Quarteto Sine Qua Non exigido pela Lei #10).

Sumário

Introdução: Como Usar Este Guia	2
1. O Ponto de Partida: Entrando na Casa e Limpando os Sapatos	2
Na Festa	2
Na Casa	3
2. A Porta da Rua: Conhecendo o Painel de Controle	3
Na Festa	3
Na Casa	3
3. O Alvará e a Fundação: Preparando o Terreno Antes de Sonhar com o Bolo	4
Na Festa	4
Na Casa	4
4. A Conversa Antes da Massa: A Entrevista que Evita Erros	4
Na Festa	4
Na Casa	5
5. A Receita na Geladeira: Especificação com Critérios Binários	5
Na Festa	5
Na Casa	5
6. A Planta do Arquiteto: O Plano Estruturado e o Grafo VSA	5
Na Festa	5
Na Casa	6

7. Escolhendo a Cozinha: A Tríade Canônica	6
Na Festa	6
Na Casa	7
8. Mãos na Massa: Salas Secretas que Nascem e Somem	7
Na Festa	7
Na Casa	8
9. A Prova Sem Mentira: Teste Primeiro, Código Depois	8
Na Festa	8
Na Casa	8
10. A Barreira da Alfândega: Juntando Tudo Sem Bagunça	8
Na Festa	8
Na Casa	9
11. O Bolo Pronto com as 4 Peças Obrigatórias	9
Na Festa	9
Na Casa	9
12. Ligando o Forno: Vendo o App Vivo no Navegador	10
Na Festa	10
Na Casa	10
13. Guardando a Receita: Salvar o Progresso sem Levar o Prato do Vizinho	11
Na Festa	11
Na Casa	11
14. Resumo das Palavras Mágicas	11

Introdução: Como Usar Este Guia

Este guia foi desenhado para ser o manual definitivo de bordo de qualquer pessoa desenvolvedora ou agente de inteligência artificial operando no **Ecossistema AIDD**. Ele elimina todas as ambiguidades do ciclo de vida de um produto de software.

Cada capítulo divide-se rigorosamente em duas falas: - **Na Festa:** A explicação intuitiva e de fácil retenção mental. - **Na Casa:** O comando real, a classe real, o arquivo no disco e o gate de verificação.

Ao final desta jornada, você terá o domínio exato de como uma ideia em linguagem natural se converte matematicamente em código testado, auditado e pronto para produção em servidores VPS.

1. O Ponto de Partida: Entrando na Casa e Limpando os Sapatos

Na Festa

Imagine que você quer fazer uma grande festa de aniversário com um bolo incrível. Mas antes de entrar na cozinha e tocar na farinha, você precisa chegar na casa certa, checar se a água e a eletricidade estão ligadas e se você tem todas as bacias e colheres limpas na bancada. Se faltar água ou o fogão estiver quebrado, não adianta tentar cozinhar.

Na Casa

O projeto todo mora em um repositório Git central (um monorepo padronizado). O primeiro passo no seu computador é baixar a casa e verificar se ela possui todos os pré-requisitos de ambiente instalados:

1. Baixar o repositório:

```
git clone https://github.com/heverton-dev/ecossistema-aidd.git
cd ecossistema-aidd
```

2. Conferir se o ambiente tem o que precisa (Preflight):

```
python ecossistema.py preflight-host
```

O que a máquina faz: Ela confere deterministicamente se você tem git, python (3.11+), node (para o frontend Next.js), docker (para os containers de infraestrutura) e os verificadores de segurança e imagens (hadolint, checkov). Se tudo estiver verde, os sapatos estão limpos.

2. A Porta da Rua: Conhecendo o Painel de Controle

Na Festa

Na entrada da cozinha existe um letreiro de madeira e um sino na porta. Você não precisa pular a janela nem adivinhar por onde entrar. Sempre que quiser algo, você fala com a porta da frente. A porta nunca dorme e sempre responde com clareza o que a casa consegue fazer.

Na Casa

A entrada única e invariante do ecossistema é o arquivo `ecossistema.py`, localizado na raiz do projeto. Ele é a CLI agnóstica que funciona em qualquer sistema operacional (Windows, Linux, macOS) e com qualquer harness de IA (Claude, Gemini, Cursor, OpenCode, Mimocode).

- **Ver o painel geral:**

```
python ecossistema.py help
```

- **Ver o status de saúde da casa hoje:**

```
python ecossistema.py status
```

- **Sincronizar as ferramentas em todos os programas de IA:**

```
python ecossistema.py components sync --tipo todos
```

3. O Alvará e a Fundação: Preparando o Terreno Antes de Sonhar com o Bolo

Na Festa

Antes de qualquer festa acontecer, alguém precisa comprar o terreno, pedir o alvará na prefeitura e pregar as regras da casa na parede: onde fica a saída de emergência, quem tem a chave de cada cômodo, que horas o barulho tem que parar. Ninguém decora o bolo nem escreve o cardápio nessa hora — essa etapa não cozinha nada. Ela só garante que, quando a cozinha entrar em ação daqui a pouco, ninguém vai construir em cima de um terreno sem escritura.

Na Casa

Essa etapa é a **aidd-forge** — a fundação e governança suprema do ecossistema. Ela nunca gera código de aplicação; ela injeta a infraestrutura de regras no projeto alvo.

- **Como acionar sozinho** (fora de um fluxo completo, só para inspecionar):

```
python ecossistema.py forge init <pasta-do-projeto> --force
```

- **O que ela realmente grava no disco** (saída real do comando — código-fonte em `tools/aidd-forge/aidd_forge/cli.py:59-74`):
 - Governança: `AGENTS.md/CLAUDE.md/GEMINI.md` apontando para a mesma fonte de regras.
 - As skills espelhadas em todos os harnesses de IA instalados na máquina.
 - As fases do projeto “cercadas” (Phase Fencer), para uma fase não pisar na outra.
 - Slash commands prontos (`/pure`, `/aidd-open`, `/freedom`, etc.).
 - O hook de pre-commit e os Quality Gates que vão fiscalizar todo commit futuro.

Nenhum desses itens é “a casa pronta” — são a fundação, o alvará e as regras da obra. Por isso a ordem certa é sempre **forge** → **planner** (seção 6) → **motor de construção** (seção 7), nunca o contrário: não dá para desenhar a planta baixa num terreno que ainda não existe, nem chamar um fiscal (Quality Gate) que ainda não foi instalado.

Na prática você raramente digita o comando acima à mão: quando roda `/pure`, `/aidd-open` ou `/freedom` (seção 7), o `forge init` e o `planner init` disparam sozinhos, nessa ordem, como as duas primeiras etapas do fluxo. A prova está em código, não em opinião — abra `scripts/orquestrador_sincrono.py:544-545` e leia literalmente (“Etapa 1: aidd-forge”, ...) antes de (“Etapa 2: aidd-planner”, ...).

4. A Conversa Antes da Massa: A Entrevista que Evita Erros

Na Festa

Você quer um bolo. Mas alguém já perguntou se o bolo é de chocolate ou de cenoura? Alguém tem alergia a leite? Quantas pessoas vão comer? A vela acende com fósforo ou pilha? Existe uma pessoa na casa chamada *Entrevistadora*. Ela não coloca a mão na massa. Ela senta com você numa mesa e faz perguntas profundas e desconfortáveis até não sobrar nenhuma dúvida ou

premissa oculta. Isso impede que alguém passe o dia inteiro assando um bolo que ninguém pode comer.

Na Casa

Esse passo é acionado pelo comando `/aidd-grill` (ou `/aidd-grill-docs` se quiser que ele leia o `MEMORY.md` e as leis do ecossistema antes de perguntar).

- **Como acionar na conversa:** Basta dizer ao assistente: `> /aidd-grill` Quero criar um aplicativo de clínica veterinária para agendamento de consultas e controle de vacinas.
 - **O que acontece por dentro:** O assistente ativa o cartão `componentes/compartilhado/skills/aidd-grill/SKILL.md`. Ele não gera código de imediato. Ele investiga:
 - Casos de borda (O que acontece se o cliente tentar agendar dois animais no mesmo horário?).
 - Invariantes de negócio (O valor da vacina pode ser negativo? Não).
 - Limites de contexto e integrações externas obrigatórias.
-

5. A Receita na Geladeira: Especificação com Critérios Binários

Na Festa

Depois que a entrevistadora tirou todas as dúvidas, ela chama a *Escriba*. A escriba pega uma folha branca e escreve a receita do bolo em letras garrafais, colando com um ímã na porta da geladeira. Na receita não está escrito “o bolo tem que ficar bem gostoso e bonito” (porque bonito é opinião subjetiva). Está escrito: “O bolo tem 3 andares, massa de chocolate, recheio de morango e 20 velas azuis”. Qualquer fiscal que olhar consegue responder sim ou não. Passou no teste!

Na Casa

O comando `/aidd-spec` pega tudo o que foi conversado no grill e transforma em uma especificação técnica formal e determinística.

- **Como acionar:** `> /aidd-spec`
 - **O que é gerado:** Um documento estruturado com critérios de aceitação binários (PASS ou FAIL).
 - Exemplo: `POST /api/v1/consultas` retorna 201 Created quando os dados forem válidos e 422 Unprocessable Entity quando a data for no passado.
 - Não há espaço para adjetivos vagos como “sistema intuitivo” ou “rápido”. Há métricas exatas de contrato.
-

6. A Planta do Arquiteto: O Plano Estruturado e o Grafo VSA

Na Festa

Agora entra o *Arquiteto da Obra* — só que ele trabalha em cima do terreno que a fundação da seção 3 já deixou pronto, nunca antes. Ele pega a receita da geladeira e desenha a planta baixa da

cozinha. Ele diz: “Olha, não dá para colocar a cobertura antes da massa esfriar. Então a equipe da massa trabalha primeiro. Mas a equipe que espreme as laranjas do suco pode trabalhar ao mesmo tempo em outra bancada, porque uma não atrapalha a outra”.

Ele desenha setinhas mostrando quem espera quem. Se uma setinha fizer um círculo que nunca acaba (ciclo infinito), o arquiteto apaga e desenha de novo. Repare: ele só desenha. Quem quebra ovo e liga o forno é a cozinha da seção 7.

Na Casa

Essa etapa é operada pela ferramenta `tools/aidd-planner`, sempre depois da `aidd-forge` (seção 3) e sempre antes do motor de construção (seção 7). Ela não cria pastas nem instala governança — só desenha.

- **Assinatura real do comando** (a mesma que `scripts/orquestrador_sincrono.py:161-168` chama por baixo dos panos):

```
python ecossistema.py planner init \
  --fluxo 1 \
  --nome "Clinica Veterinaria" \
  --slug clinica-vet \
  --dominio "agendamento-veterinario" \
  --pasta ./clinica-vet
```

--fluxo aceita 1 (`aidd-pure`, antes `aidd-generator`), 2 (`aidd-open`, antes `aidd-factory`) ou 3 (`aidd-freedom`, antes `aidd-bridge`) — a mesma numeração da Tríade da seção 7. No chat, o assistente preenche esses parâmetros sozinho a partir da entrevista (seção 4) e da especificação (seção 5): basta dizer `/planner`.

- **O que ele gera:**

1. `PLANNER.json`: A lista de fatias verticais de negócio (Vertical Slices - VSA), por exemplo: `fatia-animais`, `fatia-agendamentos`, `fatia-notificacoes`.
2. Um grafo acíclico dirigido (DAG) validado pelo algoritmo de Kahn contra o schema canônico `vsa-topological-dispatch.schema.json`. Ele impede ciclos de dependência (onde a fatia A espera a B e a B espera a A).
3. Um contrato de `handoff` (`HANDOFF_PLANNER_ENGINE.json`), validado contra `handoff-planner-to-engine.schema.json`, que o motor de construção da seção 7 lê para saber exatamente o que construir. Sem esse arquivo, a construção nem começa.

7. Escolhendo a Cozinha: A Tríade Canônica

Na Festa

Na casa existem três grandes cozinhas especializadas. Você escolhe qual forno quer usar: 1.

Cozinha do Zero: Você compra sacos de trigo e quebra os ovos na hora. Tudo nasce purinho do início ao fim. 2. **Cozinha dos Blocos Prontos:** Você pega uma massa de bolo pré-fabricada de alta qualidade e só prepara o recheio especial da casa. 3. **Cozinha da Reforma:** Você trouxe

um bolo que começou a fazer na padaria da esquina, mas veio com um cadeado e ingredientes esquisitos. Você traz para cá, tira o cadeado e troca os ingredientes ruins por produtos saudáveis.

Na Casa

Esta é a **Triade Canônica** do Ecossistema AIDD. Todas as três cozinhas utilizam a mesma entrada e no final entregam o mesmo software harmonizado em monólito modular:

Fluxo	Comando de Voz	Comando no Terminal	Motor Especialista (e nome antigo)	Quando usar
Fluxo 01 — Do Zero	/pure	python ecossistema.py run-fluxo -- fluxo pure ...	aidd-pure (antes aidd- generator)	Criar software artesanal em 8 fases completas sem código de terceiros.
Fluxo 02 — Peças Prontas	/aidd-open ou / factory	python ecossistema.py run-fluxo -- fluxo open ...	aidd-open (antes aidd- factory)	Plugar motores open-source existentes (ex.: Twenty CRM, Chatwoot, Cal.com) com fatias customizadas.
Fluxo 03 — Sem Cadeado	/freedom	python ecossistema.py run-fluxo --fluxo freedom ...	aidd-freedom (antes aidd- bridge)	Libertar protótipos de Lovable, v0 ou Bolt, arrancando o vendor lock-in e migrando para PostgreSQL na VPS.

8. Mãos na Massa: Salas Secretas que Nascem e Somem

Na Festa

Quando a equipe começa a cozinhar, ninguém divide a mesma bancada ao mesmo tempo, senão um derruba a farinha do outro. A casa tem um feitiço: ela cria uma sala temporária novinha para cada cozinheiro. O cozinheiro faz a sua fatia lá dentro. Quando a fatia fica pronta, o cozinheiro leva o prato para a mesa central e a sala mágica desaparece do mapa, sem deixar sujeira no chão.

Na Casa

Isso é o **Git Worktree Efêmero**. O ecossistema nunca implementa fatias ou tickets no galho (branch) principal aberto.

- **Como funciona:** O script `tools/aidd-master/scripts/dispatch_pipeline.py` cria a worktree (o nome da branch segue o padrão `slice/<id-da-fatia>`, definido em `_create_slice_worktree`):

```
git worktree add .worktrees/fatia-agendamento -b slice/fatia-agendamento
```

- **Garantia Inviolável:** Toda a execução é protegida por um bloco `try ... finally`. Aconteça o que acontecer (mesmo se o script falhar), o `finally` executa a limpeza imediata (`git worktree remove --force`), garantindo que o seu repositório permaneça sempre 100% limpo e sem branches órfãs.
-

9. A Prova Sem Mentira: Teste Primeiro, Código Depois

Na Festa

Nenhum cozinheiro pode dizer: “Olha, terminei o glacê, confia em mim que está doce”. Ele é obrigado a pegar uma colher limpa na frente de um fiscal. O fiscal experimenta: “Está azedo” (vermelho). Aí o cozinheiro coloca o açúcar exato. O fiscal prova de novo: “Agora está doce” (verde). Depois, o cozinheiro limpa a borda da tigela sem alterar o gosto (refatoração).

Dizer que o glacê funciona sem dar a colher para o fiscal provar é motivo de expulsão imediata da cozinha.

Na Casa

Esta é a **Lei #5 (Zero Mocks / Zero Stubs)** e a **Lei #13 (Todo Portão Deve Provar que Morde)**.

- **A skill /aidd-tdd:**
 1. **Red:** Escreve um teste real (ex.: `test_criar_consulta.py`) que chama a função de verdade e falha (exit code 1).
 2. **Green:** Escreve o código mínimo necessário em Python para a rota funcionar e o teste passar (exit code 0).
 3. **Refactor:** Limpa a estrutura, aplica tipagem estrita e melhora o desempenho mantendo o teste verde.
 - **Proibição de Código Falso:** Funções com `pass`, `return True` sem lógica real, ou `TOD0:` implementar mais tarde são imediatamente barradas pelos analisadores de sintaxe (AST).
-

10. A Barreira da Alfândega: Juntando Tudo Sem Bagunça

Na Festa

Quando a fatia do bolo sai da sala secreta, ela passa por uma roleta com guardas de terno e distintivo. O cozinheiro disse na receita que ia mexer apenas nas maçãs. Se os guardas olharem a

bandeja dele e virem que ele mexeu no vidro de canela sem autorização, a roleta trava, apita alto e devolve a bandeja. Ele só tem permissão de colocar na mesa exatamente o que pediu licença para fazer.

Na Casa

Esse é o papel do motor `vsa_join_barrier.py` e dos **51 Quality Gates** globais do ecossistema.

- **A Barreira de Junção (Join Barrier):** Lê os arquivos alterados no git diff da worktree e compara com o escopo autorizado da fatia: só `src/slices/<fatia>/`, `tests/` e `docs/`. Se o desenvolvedor ou a IA tocou em arquivos de outro módulo ou do núcleo compartilhado sem autorização, o merge é sumariamente bloqueado.
- **A Auditoria de Portões:** Antes de selar o commit, o comando `python ecossistema.py audit` roda os 40 guardas ligados ao pre-commit. Os outros 11 só rodam quando alguém os chama (1 é manual de propósito e 10 ainda não foram ligados ao commit), então um audit verde não prova que eles passaram:
 - `G_TESTES_REAIS`: Roda de verdade o pytest de cada uma das 8 ferramentas (2.343 testes coletados em 25/09/2026).
 - `G_CONTRACT_ROT`: Confere se as rotas do código batem 100% com o OpenAPI.
 - `G_SEGREDOS`: Confere se nenhuma senha ou chave vazou no código.
 - Se todos derem `exit 0`, a fatia se junta à branch principal (`main`).

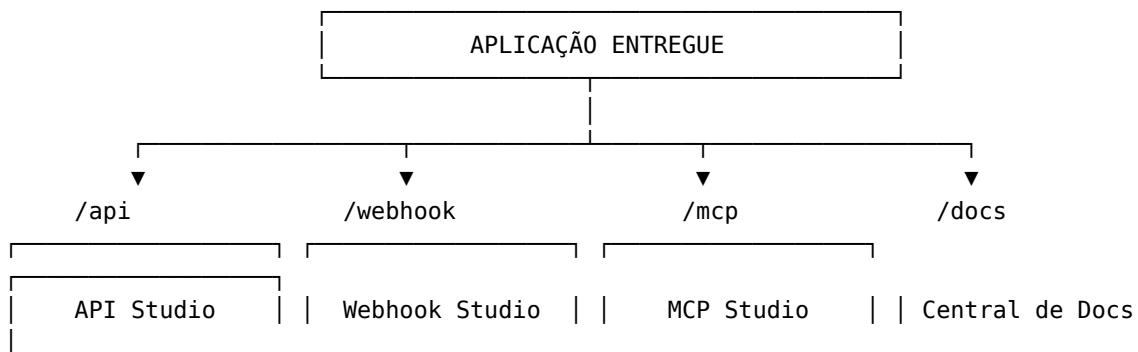
11. O Bolo Pronto com as 4 Peças Obrigatórias

Na Festa

O bolo está pronto e decorado na mesa central. Mas na nossa festa o bolo não é só para comer. Ele vem numa caixa especial com quatro acessórios que nenhuma outra padaria entrega: 1. Um livrinho ensinando cada sabor que está ali dentro. 2. Uma campainha do lado de fora da caixa para os vizinhos mandarem mensagens. 3. Um fone de ouvido para falar com robôs e máquinas inteligentes. 4. Um manual simples ilustrado para as crianças entenderem como cortar.

Na Casa

Esta é a **Lei Inviolável #10 (Quarteto Sine Qua Non Dinâmico)**. Nenhum software é considerado pronto no ecossistema se não oferecer nativamente os 4 pilares:



(OpenAPI/Swagger)	(Eventos Externos)	(Agentes e LLMs)	(Manual Humano)

1. **API Studio (/api):** Documentação interativa de 100% dos endpoints em OpenAPI 3.1 / Swagger Studio com interface gráfica viva.
2. **Webhook Studio (/webhook):** Painel para cadastrar, simular e monitorar webhooks e eventos do sistema.
3. **MCP Studio (/mcp):** Servidor Model Context Protocol nativo que permite a qualquer agente de IA interagir com as funções do app.
4. **Central de Documentação e Guia (/docs):** Manual didático escrito para humanos entenderem o propósito e a operação de cada tela do sistema.

12. Ligando o Forno: Vendo o App Vivo no Navegador

Na Festa

Chegou a hora tão esperada! O salão está iluminado, as pessoas estão sentadas, o aniversariante chega e a música toca. Você liga o interruptor e todas as lâmpadas acendem ao mesmo tempo. O bolo é servido, as pessoas usam os pratos, e tudo funciona sem engasgos.

Na Casa

Conforme a **Lei #11 (Padrão-Ouro de Stack Tecnológica)**, todo projeto gerado entrega a mesma combinação harmoniosa: - **Backend:** Python puro + SQLite WAL (modo de gravação ultrarrápido). - **Frontend:** Next.js + TypeScript + Tailwind CSS (gerado automaticamente a partir dos contratos pelo exportador `nextjs_exporter.py`).

Para colocar o seu aplicativo no ar em menos de 2 minutos:

1. Iniciar o Backend Python:

```
cd meu-projeto/backend
python server.py
# Servidor ativo em: http://localhost:8000
```

2. Iniciar o Frontend Next.js:

```
cd meu-projeto/frontend
npm install
npm run dev
# Interface ativa em: http://localhost:3000
```

3. Abrir e comemorar:

- Acesse `http://localhost:3000` para usar a aplicação completa com visual moderno e responsivo.
- Acesse `http://localhost:8000/api` para ver e testar todos os endpoints no OpenAPI / Swagger Studio.

- Acesse <http://localhost:8000/docs> para consultar a Central de Documentação e Guia.
 - Acesse <http://localhost:8000/webhook> para o Webhook Studio e simulação de eventos.
 - Acesse <http://localhost:8000/mcp> para conectar seu assistente Claude ou Gemini como operador do sistema.
-

13. Guardando a Receita: Salvar o Progresso sem Levar o Prato do Vizinho

Na Festa

No fim do dia, alguém tira foto de tudo o que ficou pronto e guarda no álbum da casa. O fotógrafo é rápido: fotografa a mesa inteira de uma vez. O problema é quando dois cozinheiros dividem a mesma mesa. A foto sai com o bolo de um e a torta do outro, e a legenda só fala do bolo.

Na Casa

O “fotógrafo” é o **faz-commit**. Digitado no Git Bash, ele executa sempre o `scripts/faz_commit.py` do repositório (desde 24/09/2026, o comando global é só um atalho para ele):

1. `git add -A` — coloca **tudo** o que mudou na pasta no commit.
2. Gera a mensagem a partir do diff (por IA, se houver chave de API; senão, pergunta para você).
3. Faz o commit, que dispara os guardas do pre-commit, cada um mostrado em **uma linha** com o andamento ao vivo (`-v` mostra tudo).
4. Faz o push.

Se um guarda reprovar, o resumo final diz a etapa, o guarda, o arquivo:linha e o comando exato para repetir só aquele ponto. `faz-commit --dry-run` simula tudo e devolve o stage exatamente como estava.

Cuidado com duas sessões de IA na mesma pasta: como o passo 1 leva tudo, o trabalho da outra sessão entra junto. Isso aconteceu de verdade em 24/09/2026 (commit `a1ea899`). Com mais de uma sessão aberta, prefira `git add <arquivos>` e `git commit` só do que é seu.

Para saber depois qual sessão fez o quê, registre cada uma com `python ecossistema.py sessao registrar` (ou `/aidd-sessao`). O ID vai para `secoes/historico_sesoes.json` e para o índice legível `secoes/INDICE-SESSOES.md`; `sessao listar` e `sessao buscar` consultam.

14. Resumo das Palavras Mágicas

Se você precisar de uma folha de cola rápida para lembrar o que dizer em cada momento — a ordem das linhas abaixo é a ordem real de execução, não é só didática:

Momento	O que falar no chat	O que rodar no terminal
Conferir o terreno	<i>“Qual o status da casa?”</i>	python ecosystema.py status
Fazer as perguntas difíceis	/aidd-grill <sua ideia>	—
Escrever o contrato certo	/aidd-spec	—
Instalar a fundação e a governança	<i>(roda sozinho dentro de /pure, /aidd-open, /freedom)</i>	python ecosystema.py forge init <pasta> -- force
Desenhar a planta de fatias	/planner	python ecosystema.py planner init --fluxo N -- nome ... --pasta ...
Construir do Zero Puro	/pure	python ecosystema.py run-fluxo --fluxo pure
Construir com Motores Abertos	/aidd-open ou /factory	python ecosystema.py run-fluxo --fluxo open
Libertar app do Lovable/v0	/freedom	python ecosystema.py run-fluxo --fluxo freedom
Despachar as fatias em worktrees	/dispatch	python ecosystema.py dispatch --planner PLANNER.json
Passar pelos guardas	<i>“Rode os testes”</i>	python ecosystema.py audit
Anotar a sessão	<i>“Registre esta sessão”</i> ou / aidd-sessao	python ecosystema.py sessao registrar --id <ID> --harness claude -- titulo "..."
Registrar a vitória	<i>“Salve o progresso”</i>	faz-commit (no Git Bash)

Este livro é um documento vivo do Ecossistema AIDD. Ele prova que rigor de engenharia, governança matemática e explicação simples podem morar exatamente sob o mesmo teto.